SystemVerilog 검증 수업 정리

2026.11.09 (월) · 1~6교시 · 카운터 랜덤 제약 리뷰, RAM 테스트벤치 v2/v3, 시험·발표 일정
출처: `1109class.txt` 녹음본, `ram.sv`, `tb_ram_v2.sv`, `tb_ram_v3.sv`

> **오늘의 결론.** 검증 구조에 정답은 하나가 아니다. 무엇을 랜덤으로 만들고, 언제 드라이브·샘플·판정하는지 스스로 계획하고 타이밍으로 설명할 수 있어야 한다.

꼭 챙길 일정	날짜	내용
핸드라이팅 시험	11/16 (월)	2시간, 개념 + 설계 문제 (6장 참고)
UVM 검증 발표	11/23 (월)	기존 설계를 UVM으로 검증하고 시나리오·흐름 발표

교시	다룬 내용
1교시	카운터 숙제 리뷰(3사이클 이상 연속 run/stop), randc·dist, 발표 주제 논의
2교시	일정·시험 범위·로드맵·면접 조언 → SRAM 설계, 검증 시나리오 시작
3교시	스코어보드 판정 시점, X 처리, 블록 다이어그램 설계, 이벤트 타이밍
4교시	Expected를 만드는 두 가지 구조 비교, TB 코딩 시작
5교시	실행·디버그(이벤트 누락으로 드라이버 밀림), 리포트 추가, NBA 이벤트
6교시	Generator→Scoreboard 메일박스 추가, 매 사이클 판정 구조를 숙제로 제시

1. 카운터 TB 리뷰: 3사이클 이상 연속 run/stop

핵심은 값이 아니라 **유지 횟수를 랜덤 변수로 만드는 것**이다. 단순 랜덤에 맡기면 3회 이상 연속은 확률일 뿐 보장되지 않는다.

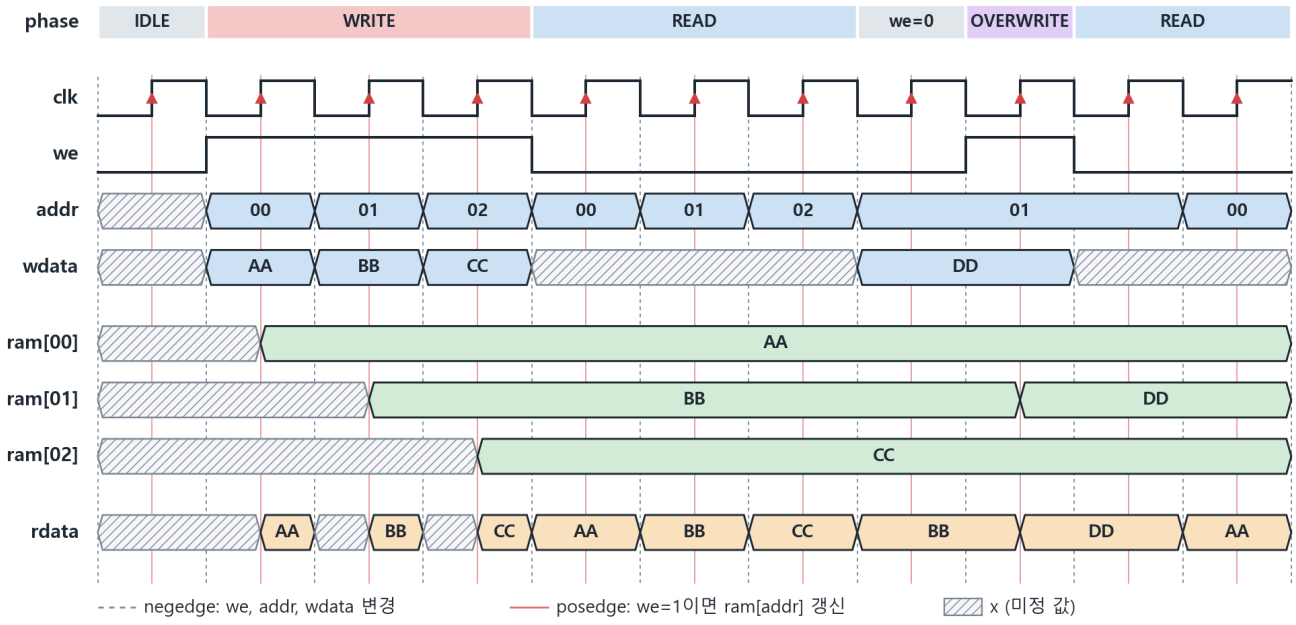
결정 사항	방법	이유
기준 신호	`run_stop` 하나를 먼저 기준으로 잡음	카운터 동작을 좌우하는 신호. 여러 개를 한 번에 하면 복잡해짐
유지 횟수	transaction에 `rand int unsigned run_stop_hold` 추가	음수가 나오면 안 됨. 메시지 하나에 반복 횟수를 함께 실어 보냄
범위	`inside {[3:16]}`	처음 256을 생각했으나 값 변화를 충분히 보기 위해 16으로 축소
반복 위치	Generator는 1회 생성, Driver가 반복 후 이벤트	Generator가 같은 메시지를 여러 번 보내면 조건이 복잡해짐. Gen–Drv는 이미 메일박스+이벤트로 동기화됨
Driver 흐름	첫 드라이브 1회 후 나머지 `hold-1`회를 negedge마다 반복	메시지를 읽을 때마다 negedge를 잡으면 매번 1클럭을 손해 봄
확률 조절	clear 등은 `dist`로 0:9, 1:1	clear는 가끔 발생하는 편이 카운트 동작을 보기 좋음

구분	`rand + dist`	`randc`
동작	지정한 가중치로 값 선택	범위 안의 값을 한 번씩 순환
1비트 결과	가중치 그대로 조절 가능	두 번 중 0과 1이 한 번씩 → 5:5
주의	횟수가 적으면 비율 오차 발생	**`dist`와 함께 사용 불가**
오늘 사용	`reset`, `mode`, `clear`	`run_stop`, RAM의 `we`

2. RAM(SRAM) 설계와 검증 시나리오

항목	내용
구조	64 × 8비트, `addr[5:0]`, `wdata[7:0]`, `we`, `clk`, `rdata[7:0]`, 리셋 없음
쓰기	`we=1`일 때 clk 상승 엣지에서 `ram[addr]`에 저장
읽기	오늘은 **조합 출력**: `addr`가 바뀌면 클럭 없이 `rdata`가 바로 변함
순차 출력	상승 엣지 이후 `rdata`가 나오므로 조합 출력과 같은 시나리오를 쓸 수 없음. 타이밍을 다시 계획해야 함
랜덤 대상	`addr`, `wdata`, `we` 모두 랜덤 (`we`는 `randc`로 5:5)
검증 목표	① 모든 주소의 쓰기/읽기 ② `we=1`일 때만 저장 ③ 조합 출력 동작 확인

RAM 64x8 타이밍도 (동기 쓰기 / 조합 읽기)



ram_timing.png: 입력은 하강 엣지에서 변경, 저장은 상승 엣지, rdata는 addr와 메모리 변화에 즉시 반응

3. 타이밍과 스코어보드 설계

질문	수업 결론	이유
언제 드라이브?	negedge	상승 엣지 전에 입력을 안정시킴
언제 샘플?	드라이브 이후. posedge, negedge, 지연 후 중 설계자가 선택	"드라이브 이후"만 필수이고 시점은 계획의 문제
모니터 범위	`we` 값과 상관없이 매번 스코어보드로 전달	`we`도 랜덤이라 언제 쓰는지 스코어보드가 알아야 함
쓰기 직후 판정	상승 엣지 순간에는 판정 불가, 그 이후라면 가능	상승 엣지 시점의 값은 저장 전. 최소한의 지연 후에 확인
Expected 저장소	스코어보드에 64칸 배열	주소로 바로 접근. 큐는 FIFO 성격이라 이 용도엔 불편
배열 타입	**`byte`가 아니라 `logic`**	2-state면 미기록이 0이 되어 "0을 쓴 경우"와 구분 불가. `logic`은 X로 시작
비교 연산	`===`	X까지 포함해 비교해야 미기록 주소 처리 가능

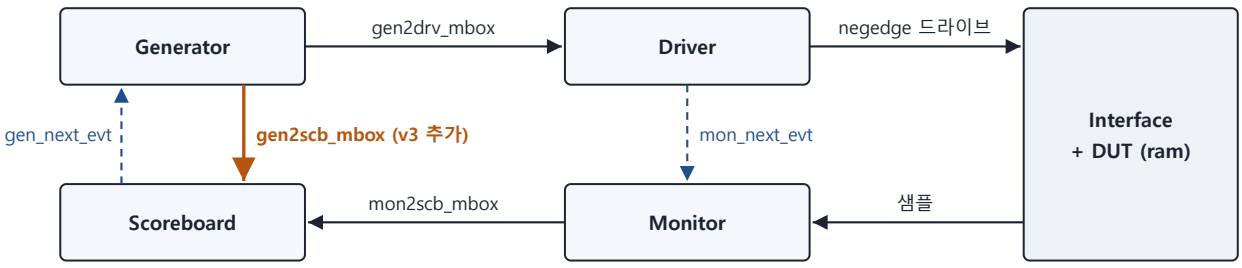
이벤트 레이스 컨디션

Driver가 드라이브 직후 같은 시각에 이벤트를 날리면, 받는 쪽이 아직 대기 중이 아닐 때 이벤트를 놓친다. 5교시에는 이벤트 없이 상승 엣지만 기다린 Monitor가 먼저 돌아 Driver가 한 칸 밀렸고, `mon_next_evt`를 추가해 해결했다.

회피 방법	동작	비고
`#1` 지연 후 이벤트	시간차로 레이스 자체를 피함	수업에서 가장 편한 방법으로 언급
`->>` (넌블로킹 이벤트)	NBA 영역에서 이벤트 발생	`tb_ram_v2`
`wait(ev.triggered)`	같은 time step에 이미 발생한 이벤트도 감지	`tb_ram_v2`
`->` + `@(ev)`	받는 쪽이 먼저 `@`에서 대기 중이어야 안전	`tb_ram_v3`: Monitor가 이전 사이클부터 대기

이런 이벤트 제어는 UVM에서는 직접 하지 않아도 되며, 조건만 잘 걸면 된다.

4. tb_ram_v2와 tb_ram_v3 구조



실선: 메일박스/신호, 파선: 이벤트, 주황: v3에서 추가된 경로

항목	tb_ram_v2	tb_ram_v3
수업상 방식	Monitor 값만으로 판단, 스코어보드 버퍼 사용	Generator 값도 스코어보드로 전달해 2개를 비교
메일박스	gen2drv, mon2scb (2개)	gen2drv, mon2scb, gen2scb (3개)
Drv→Mon 이벤트	`->>` + `wait(.triggered)`	`->` + `@`
샘플 시점	posedge	posedge + `#1`
`we=1` 처리	Expected 저장만, 판정 없음	wdata 판정 후 Expected 저장
`we=0` 처리	Expected `===` rdata	Expected `===` rdata
판정 빈도	읽기 사이클만	매 사이클
리포트	전체 pass/fail, read/write 개수	전체 + read·write 각각의 pass/fail
공통	`repeat(run_count)`, `join_any` 후 `report`와 `disable fork`, 트랜잭션당 1클럭 처리	

5. 실행 결과와 코드 대조 (수업 외 추가 확인)

작성한 파일을 수정 없이 Vivado 2020.2 xsim으로 시드 3개씩 실행했다. 전부 FAIL 0이지만, 아래 표처럼 **PASS의 내용**은 더 살펴볼 부분이 있다.

TB	시드	전체	쓰기	읽기	PASS	FAIL	X를 읽은 횟수	we 최장 연속	wdata 최댓값
`v2`	1	30	20	10	10	0	7 / 10	6	62
`v2`	2	30	11	19	19	0	16 / 19	6	63
`v2`	3	30	16	14	14	0	13 / 14	5	63
`v3`	1	30	14	16	30	0	13 / 16	5	58
`v3`	2	30	13	17	30	0	14 / 17	3	63
`v3`	3	30	19	11	30	0	11 / 11	7	57

확인 사항	현재 상태	보완 방향
① v3 쓰기 판정 대상	Generator의 wdata와 Monitor의 wdata를 비교. TB 신호 전달만 확인하며 DUT 저장 결과는 보지 않음	`#1` 후 로그의 rdata에 이미 쓴 값이 보이므로 (16ns: wdata 20, rdata 20), rdata를 `===`로 비교해야 쓰기 검증이 됨
② 읽기 PASS의 대부분이 X	주소 64개에 30회 실행이라 읽기 대부분이 미기록 주소의 X `===` X	실행 횟수를 늘리거나(수업은 100회) 주소 범위를 좁혀 "쓴 값을 다시 읽는" 경우를 늘림
③ `randc` 순환 초기화	매번 `tr = new()`로 객체를 새로 만들어 순환 상태가 유지되지 않음. 표의 `we` 최장 연속이 2를 넘음	5:5 보장이 필요하면 같은 객체를 재사용해 randomize. 수업의 "정확히 5:5가 안 되더라"의 원인일 수 있음
④ wdata 범위	8비트인데 `inside {[0:63]}`로 제한되어 상위 2비트가 항상 0	`addr` 제약을 복사한 것으로 보임. 0~255로 넓혀야 전체 비트 검증
⑤ 트랜잭션 공유	같은 `tr` 핸들을 두 메일박스에 전달	수업 권고: 안전하게 복사본(TR2)을 따로 만들어 전달

6. 일정, 시험, 로드맵

시기	내용
11/9~10	SV로 RAM, FIFO 마무리 예상
11/11~13	UVM 전에 Synopsys 툴 학습, SV 추가 실습 *
11/16 (월)	핸드라이팅 시험
11/16 주	UVM 실습: adder → 플립플롭 → 카운터 → RAM → FIFO → (가능하면) UART
11/23 (월)	UVM 검증 발표
11/23 이후	RISC-V RV32I 약 3~4주: 버스, GPIO·UART·FND 등 주변장치, FPGA에서 SW 실행, UVM 검증
12월~1월	C언어·임베디드(간단히, 어셈블리와 연계)
2월~3월	영상 처리·AI 등 플러스 알파, 최종 팀 프로젝트(약 4주)
3월 말	최종 발표. 우수 2~3팀은 기업 대상 외부 발표 예정(계획 변동 가능)

11/23 발표	내용
주제 후보	① 10,000진 카운터 (FSM + 데이터패스 중심, 버튼 디바운스 제외, FND는 제외 쪽으로 논의) ② UART + FIFO + ASCII 디코더
방식	설계는 기존 것을 그대로 쓰고 **검증 과정**을 UVM으로 구현해 발표. SV 발표는 하지 않음
발표 포함	시나리오, 랜덤화 대상과 제어 방법, 흐름도, 시뮬레이션 결과

11/16 시험	내용
시간·형식	2시간. 개념 문제 + 설계 코딩 문제 (시간이 빠듯하다고 안내)
개념	SV 문법 시험은 특별히 없음. 시뮬레이션 레이스 컨디션은 출제 가능
설계	복잡하지 않은 수준: MUX, 디코더, ALU, 비교기, 카운터, 플립플롭, FSM, ASM의 코드화, FIFO, UART
참고	UART 등의 스펙·타이밍은 문제에 제시. 복잡한 인터페이스는 출제하지 않음

강사 조언	내용
숙제	못 했어도 한 데까지 제출. 어떻게 생각했는지 알아야 오해를 바로잡아 줄 수 있음
면접	이력서·포트폴리오에 쓴 내용은 전부 설명할 수 있어야 함. "왜 했어?"를 반복하는 압박 질문에 "다른 친구가 해서 몰라요"는 탈락
태도	먼저 해 보고 발표를 자원. 틀려도 되니 수업 중에 묻고, 답을 받기 전에 AI 없이 먼저 고민
최종 프로젝트	주제·스펙을 팀이 직접 결정. 4주는 짧으니 역할 분담과 팀원 간 협업이 중요

7. 숙제와 다음 할 일

항목	출처 / 상태
☐ 카운터 TB: 시나리오·흐름도를 정리해 제어 방식을 설명할 수 있게 준비	1교시
☐ RAM TB 블록 다이어그램 직접 작성	3교시
☐ gen→scb 메일박스 추가 + 매 사이클 판정 구조 완성 후 이전 구조와 비교	6교시 숙제 / `tb_ram_v3.sv` 작성됨, 5장 ① 보완 권장
☐ 순차 출력 RAM의 타이밍 계획	2·4교시 언급 / 아직 조합 출력만 진행
☐ 11/16 핸드라이팅 시험 대비	6장 범위 참고

8. 녹음에서 확인이 필요한 부분

녹음 변환 오류가 많아 아래는 문맥으로 해석했다. 일정과 시험 관련 항목은 강사에게 확인하는 것을 권장한다.

녹음 표기	정리본 해석	상태
만진 카운터	10,000진 카운터	문맥상 유력
시답시토리, 시나 스토리	Synopsys 툴 (* 표시 항목)	추정
3이 아이	RV32I	문맥상 유력
에큐리 버스	RISC-V에 붙일 버스 프로토콜	**이름 불명, 확인 필요**
오회점 트리플럽	플립플롭 (종류 미상)	추정
백관식 한두 문제	시험의 개념 문제 형식	**객관식/주관식 확인 필요**
프 펙션, 앰플7	시험 범위 언급으로 보이나 복원 불가	**확인 필요**
발표 주제 최종안	녹음에서 "2가지 안을 놓고 가"까지만 확인	**미확정**
3·4교시 경계, 6교시 후반	학생 질의 일부와 끝부분 잡음은 복원 불가하여 제외	제외